

Sistemas Operativos de Tiempo Real



Técnicas Digitales III

Introducción

- Según las restricciones temporales, existen distintos tipos de aplicaciones:

- **Ofimática**
- Multimedia y video juegos
- Sistemas de control



Introducción

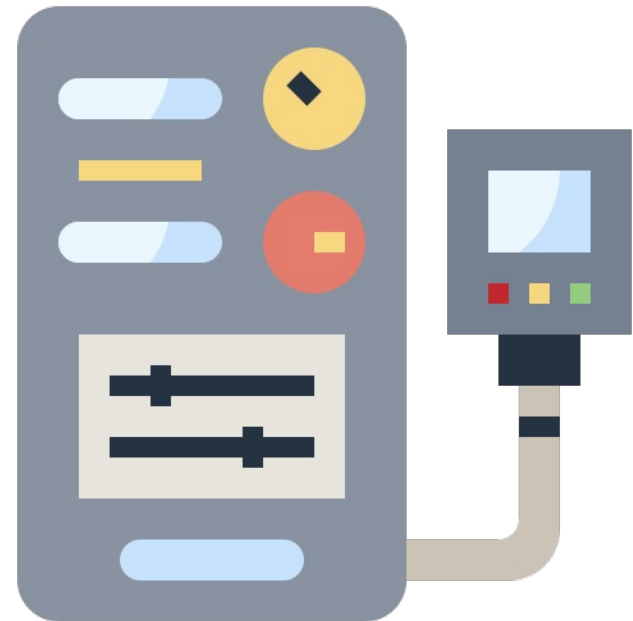
- Según las restricciones temporales, existen distintos tipos de aplicaciones:

- Ofimática
- **Multimedia y video juegos**
- Sistemas de control



Introducción

- Según las restricciones temporales, existen distintos tipos de aplicaciones:
 - Ofimática
 - Multimedia y video juegos
 - **Sistemas de control**



Introducción

- Para cada una de estas aplicaciones, se utilizan distintos tipos de Sistemas Operativos

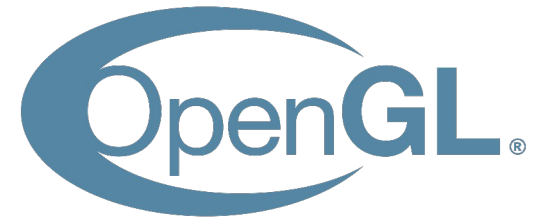
- **Sistemas operativos de propósito general**

- Extensiones multimedia
- Sistema operativo de tiempo real



Introducción

- Para cada una de estas aplicaciones, se utilizan distintos tipos de Sistemas Operativos
 - Sistemas operativos de propósito general
 - **Extensiones multimedia**
 - Sistema operativo de tiempo real



Introducción

- Para cada una de estas aplicaciones, se utilizan distintos tipos de Sistemas Operativos

- Sistemas operativos de propósito general
- Extensiones multimedia
- **Sistema operativo de tiempo real**



NuttX RTOS



VxWorks

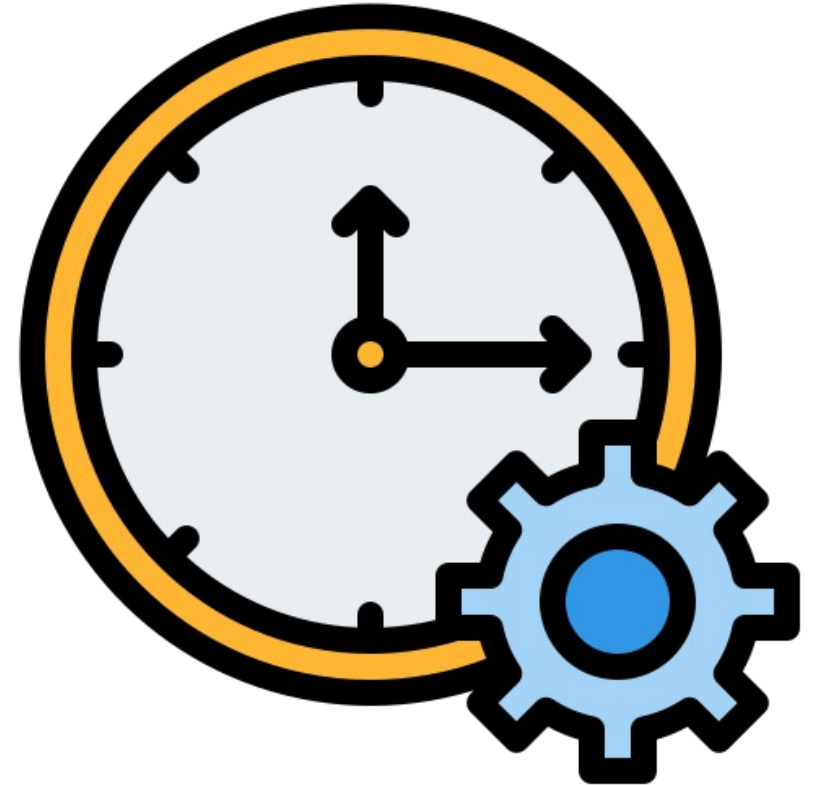
Definición

- Sistema en tiempo real:
Un sistema informático en tiempo real es aquel en el que la corrección del resultado depende tanto de su validez lógica como del instante en que se produce



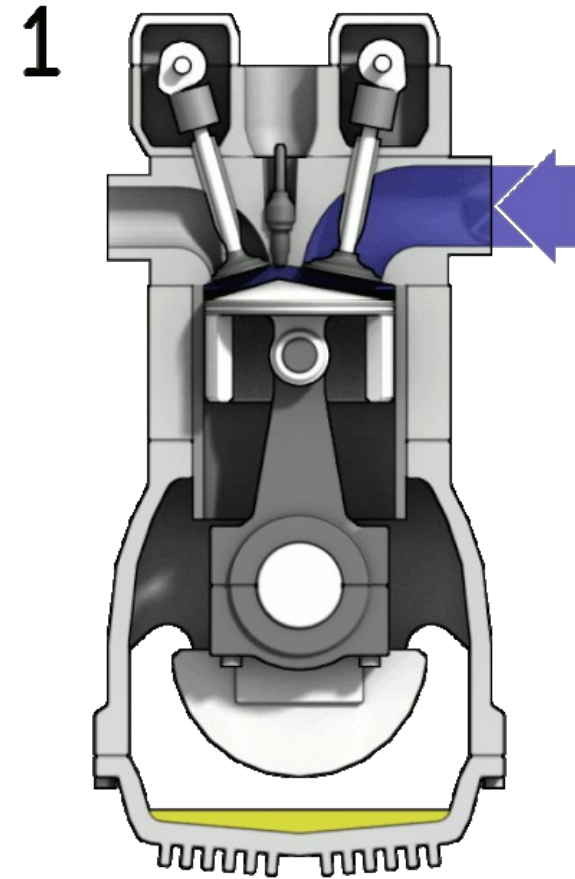
Definición

- Para garantizar el tiempo de respuesta el sistema necesita **determinismo**
- Esto implica tiempos de ejecución acotados, de forma de poder garantizar **restricciones temporales**



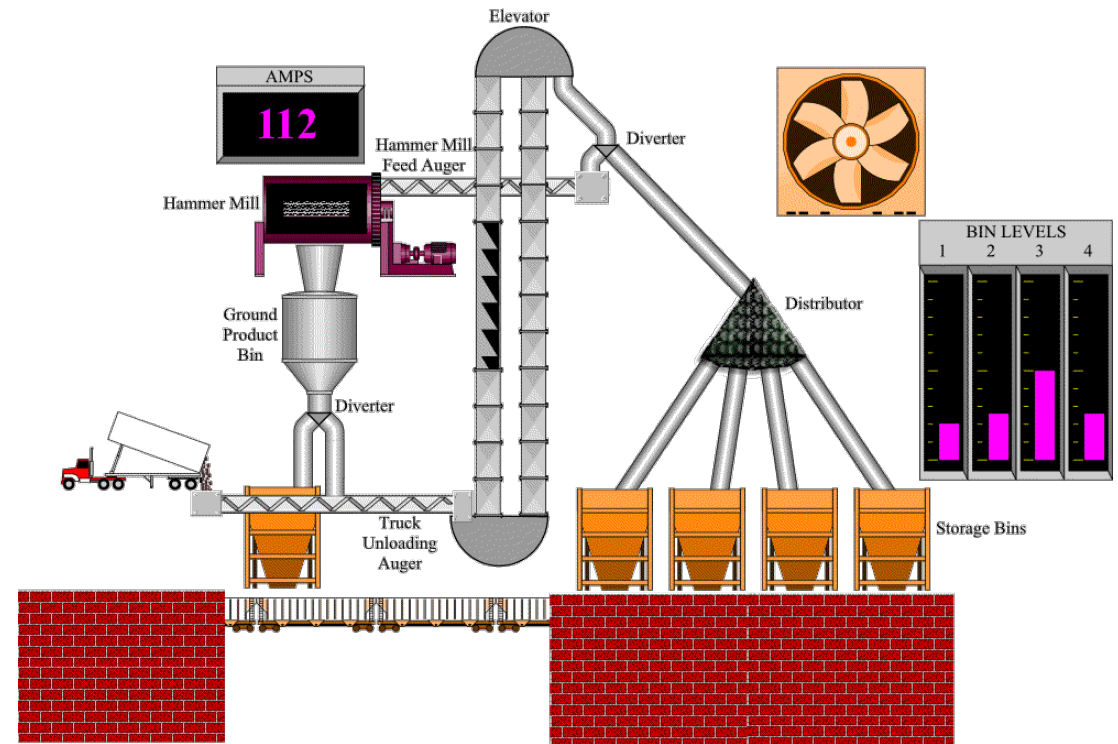
Motor de combustión interna

- Flujo de aire.
- Presión en el múltiple.
- Oxígeno.
- Temperaturas.
- Tiempo de inyección.
- Avance de encendido.



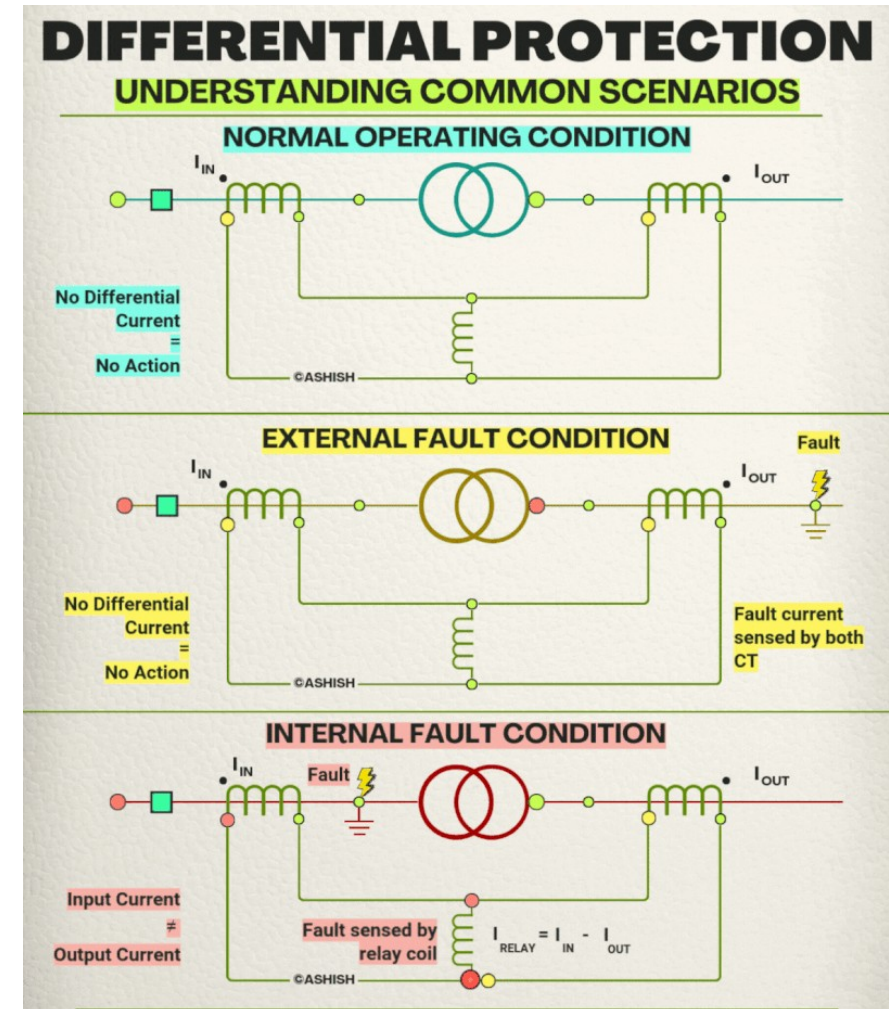
Proceso continuo

- Presión.
- Temperatura.
- Volumen.
- Tiempo de muestreo.
- Actuadores.



Sistema eléctrico

- Medición de corrientes.
- Medición de tensión.
- Evaluación de estados.
- Cálculo.
- Actuación en interruptores.



Robótica

- Sensores de posición y movimiento.
- Sensores de proximidad y distancia.
- Sensores inerciales.
- Actuadores eléctricos, neumáticos, hidráulicos.



Ejercicio

<https://www.menti.com/>

Código: 8562



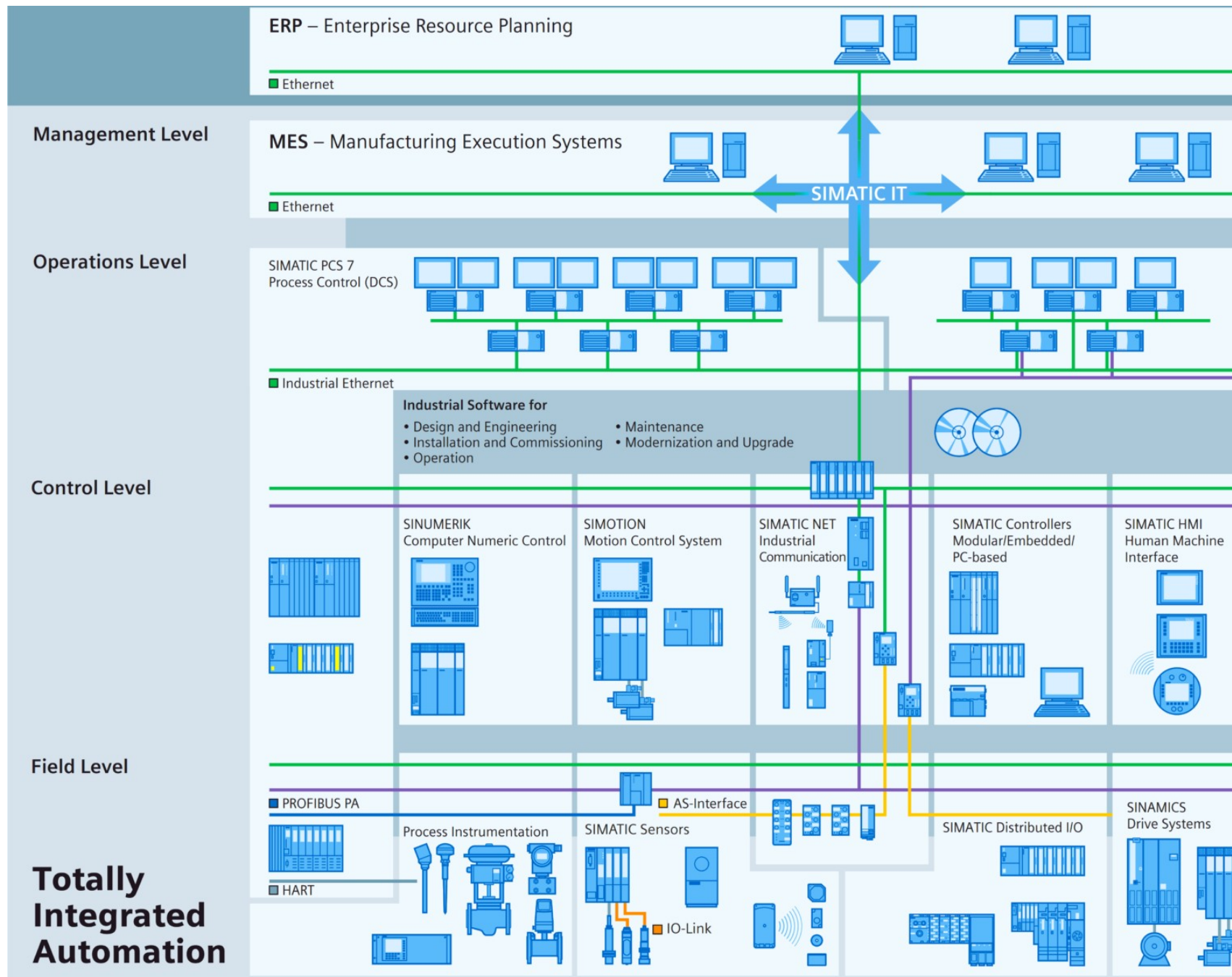
Definición

- Conviene destacar que un sistema de control en tiempo real no es necesariamente un sistema “rápido”.
- La velocidad tiene que ser tal que se ajuste a las restricciones temporales del sistema al que aplica.
- Debe ser **determinista**.



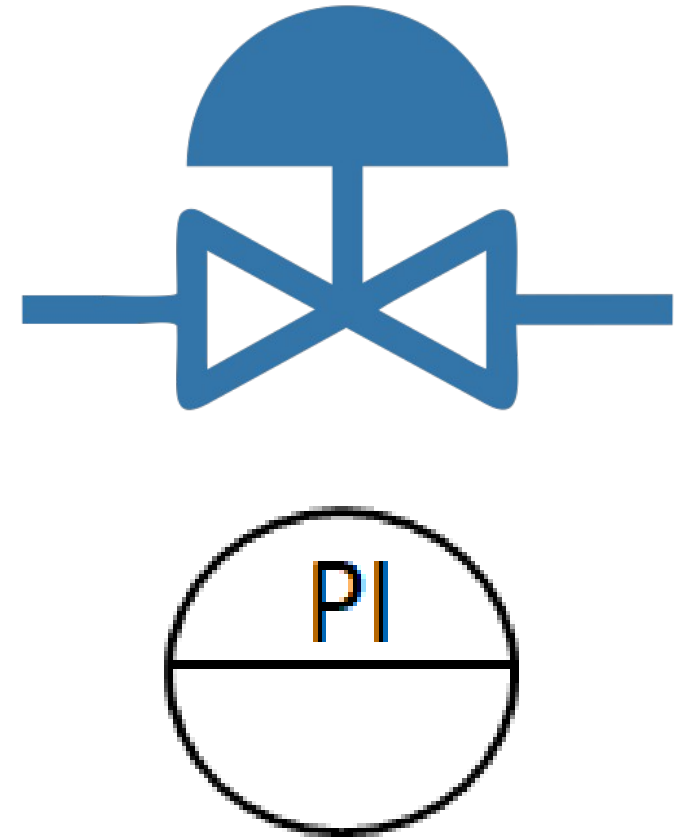
Niveles jerárquicos

- En los sistemas de control es frecuente encontrar software dividido en niveles jerárquicos:
 - Adquisición de datos y actuadores.
 - Algoritmos de control (PID).
 - Algoritmos de supervisión (trayectorias).
 - Interfaz de usuario, registro de datos.



Niveles jerárquicos

- Adquisición de datos y actuadores:
 - Este es el nivel más bajo en la jerarquía, encargado de interactuar con el hardware del sistema a controlar.



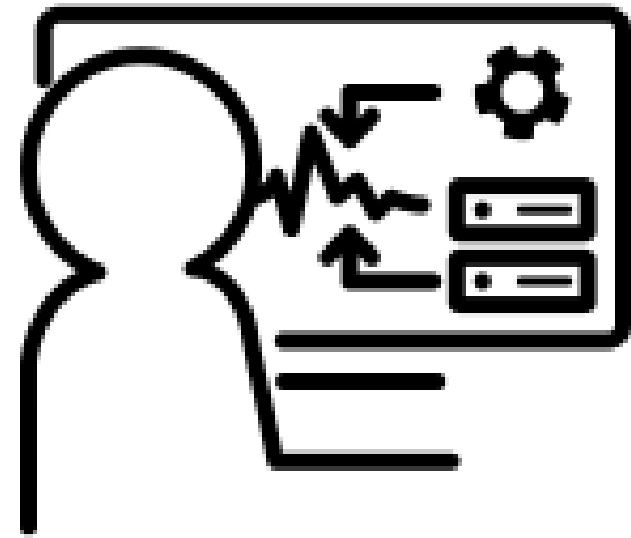
Niveles jerárquicos

- Algoritmos de control (PID):
 - Este nivel se corresponde con los algoritmos de control que, a partir de las medidas obtenidas por el sistema de adquisición de datos.



Niveles jerárquicos

- Algoritmos de supervisión:
 - En general, se encarga de supervisar el proceso y generar consignas a los controles inferiores.



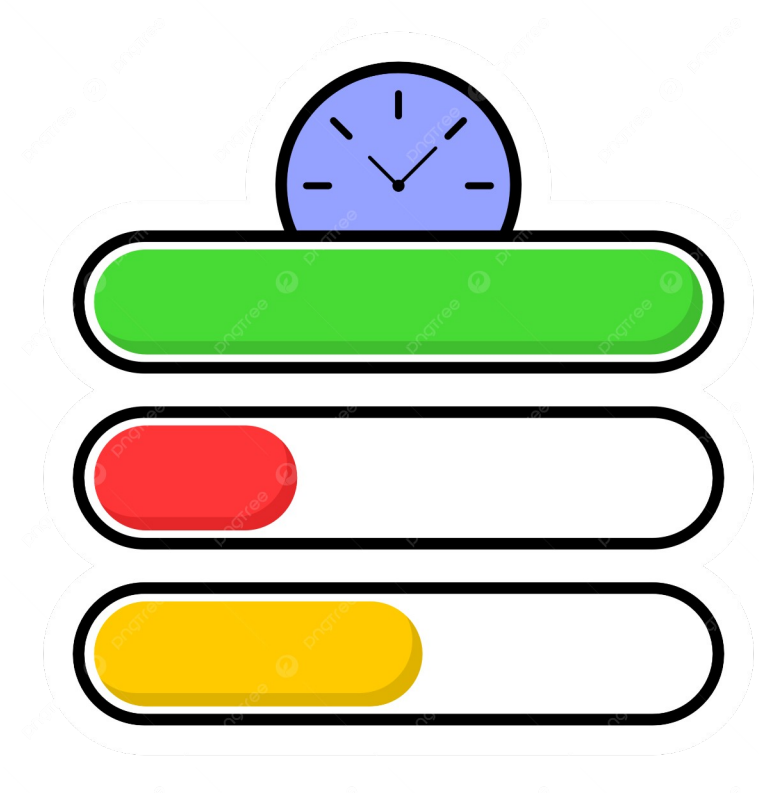
Niveles jerárquicos

- Interfaz de usuario:
 - Este nivel se corresponde con los algoritmos de control que, a partir de las medidas obtenidas por el sistema de adquisición de datos.



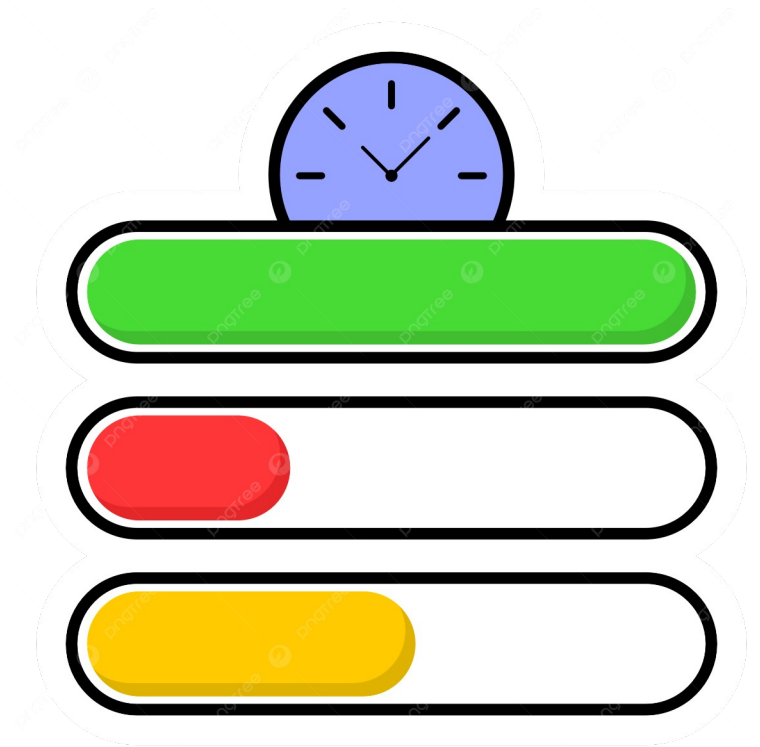
Tareas

- El paralelismo es una característica inherente a los sistemas en tiempo real.
- La ejecución se realiza de manera asíncrona, respondiendo a sucesos externos.
- NO todas las tareas son igual de urgentes. En general la criticidad está asociada a la jerarquía de control.



Tareas

- En un sistema en tiempo real el programa se divide en tareas que se ejecutan en paralelo, cada una de ellas con una determinada prioridad, para poder cumplir con sus restricciones temporales.



Tareas

- Salvo que se disponga de un sistema con varios procesadores, la ejecución en paralelo se consigue mediante la sucesión rápida de ejecuciones parciales de las tareas



Dificultades

- Son necesarias técnicas para permitir la conmutación entre las diversas tareas, respetando las restricciones temporales de cada una de ellas.
- También es necesario establecer mecanismos para el intercambio de información entre tareas.



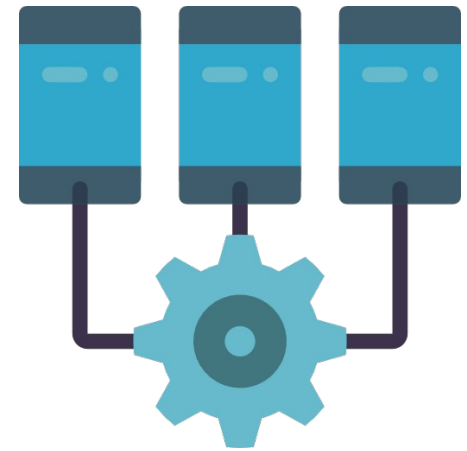
Dificultades

- La depuración de este tipo de sistemas es compleja, ya que su comportamiento no depende sólo de las entradas, sino también del instante en el que éstas cambian. Por tanto, es posible que un programa falle sólo si cambia una entrada en un instante determinado y no en otro.
- Conseguir reproducir un error de este estilo repetidamente para depurarlo es imposible.



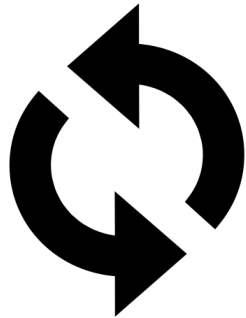
Métodos para implantar un Sistema en Tiempo Real

- Existen varias técnicas de programación para ejecutar tareas en paralelo:
 - Procesamiento secuencial (Bucle de scan).
 - Primer plano / segundo plano (Foreground/Background).
 - Multitarea cooperativa.
 - Multitarea expropiativa (preemptive).



Procesamiento secuencial (scan loop)

- Consiste en ejecutar todas las tareas consecutivamente una y otra vez dentro de un bucle infinito.



```
main()
{
    /* Inicialización del sistema */
    while(1){ /* Bucle infinito */
        Tarea1();
        Tarea2();
        /* ... */
        TareaN();
    }
}
```

Procesamiento secuencial (scan loop)

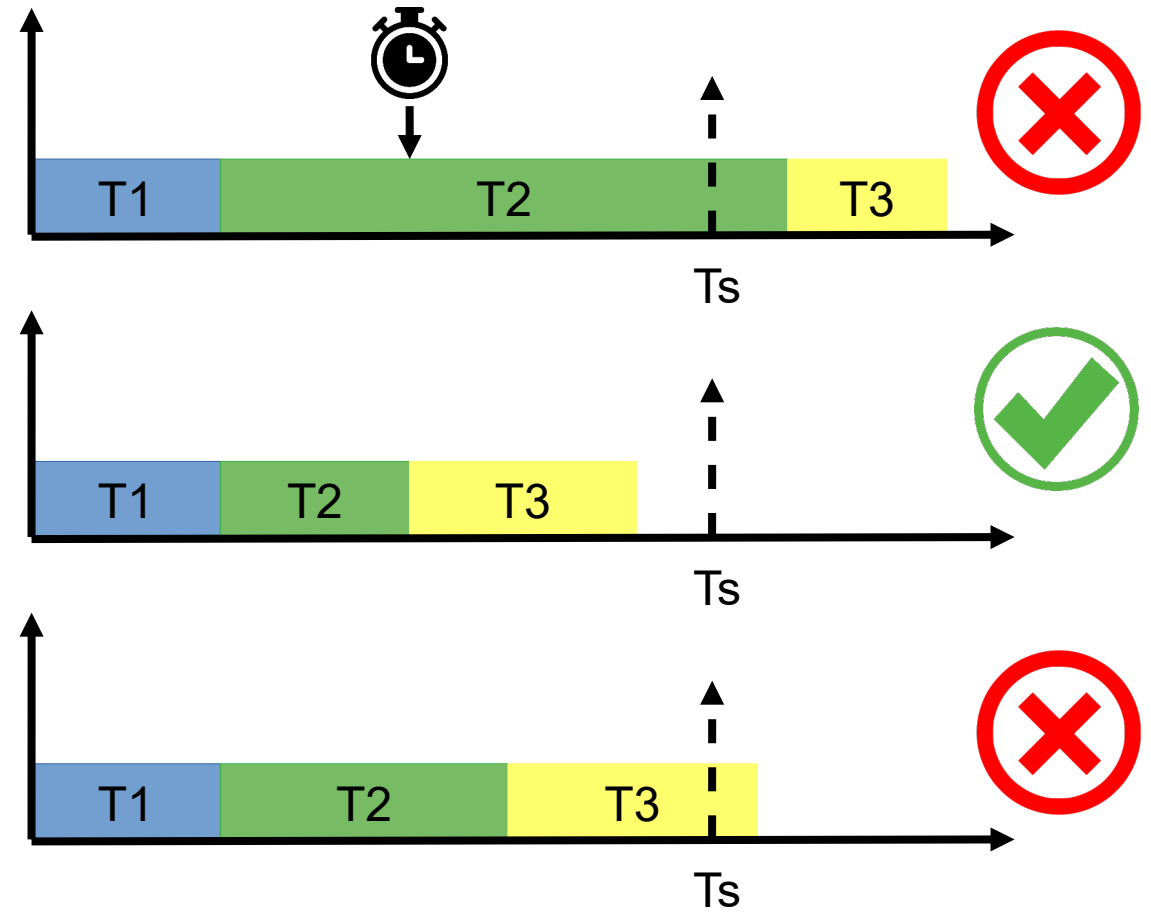
- Considerar que en un programa convencional un bucle infinito se considera un error.



```
main()  
{  
    /* Inicialización del sistema */  
    while(1){ /* Bucle infinito */  
        Tarea1();  
        Tarea2();  
        /* ... */  
        TareaN();  
    }  
}
```

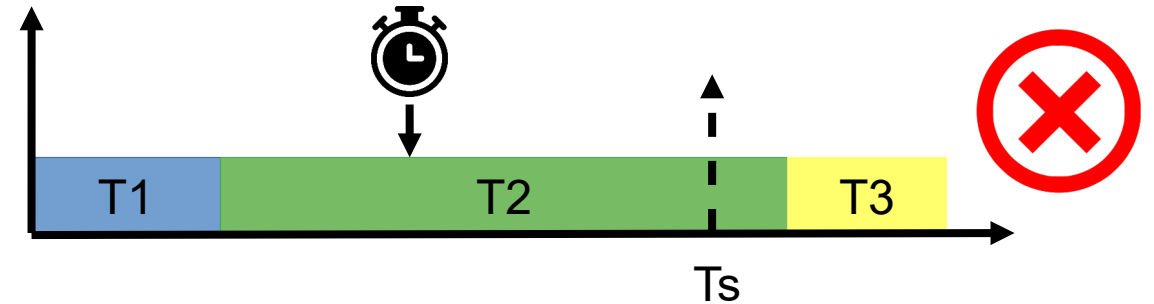
Procesamiento secuencial (scan loop)

- Condiciones:
 - Las tareas no pueden bloquearse.
 - La sumatoria de los tiempos máximos de ejecución de cada tarea debe ser inferior al período de muestreo necesario.



Procesamiento secuencial (scan loop)

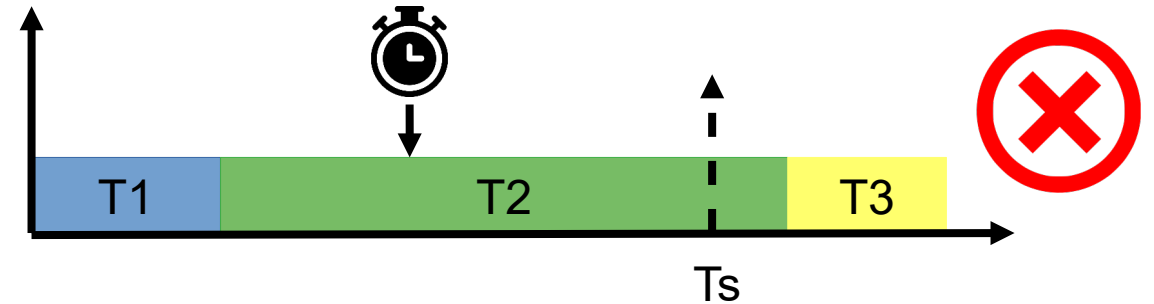
- Condiciones:
 - Las tareas no pueden bloquearse.
 - Los eventos externos son asíncronos con el funcionamiento del sistema.
 - No se puede garantizar el tiempo del scan loop.



```
main()
{
    while(1){ /* Bucle infinito */
        Tarea1();
        /* Espera un evento externo */
        while(evento_externo == FALSO);
        /* ... */
        TareaN();
    }
}
```

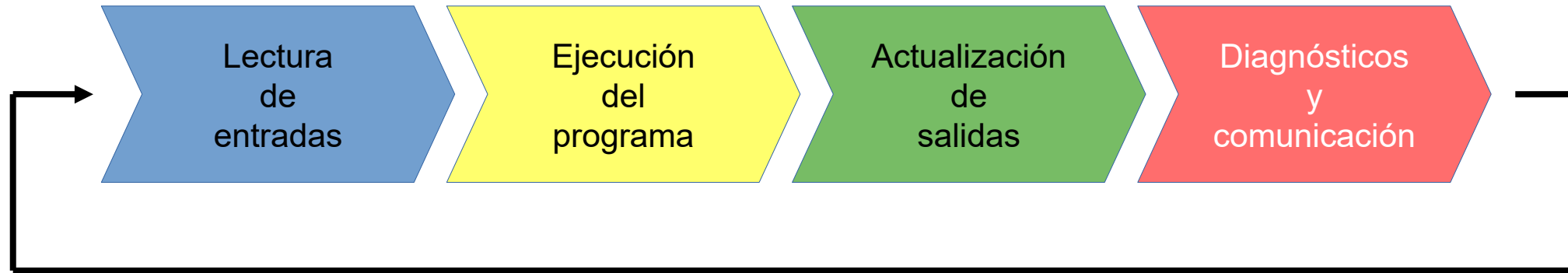
Procesamiento secuencial (scan loop)

- Condiciones:
 - Las tareas no pueden bloquearse.
 - Los eventos externos son asíncronos con el funcionamiento del sistema.
 - No se puede garantizar el tiempo del scan loop.

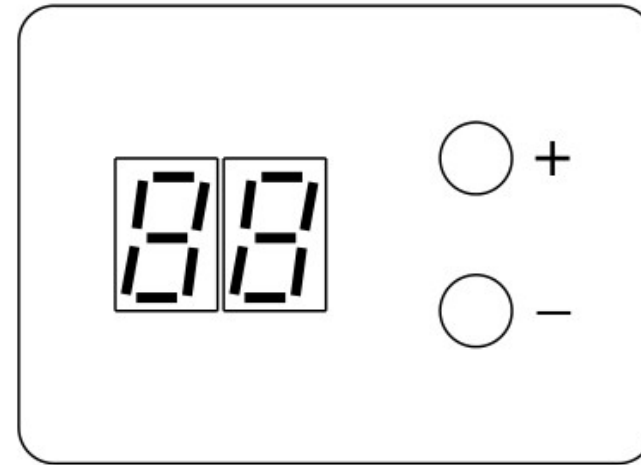
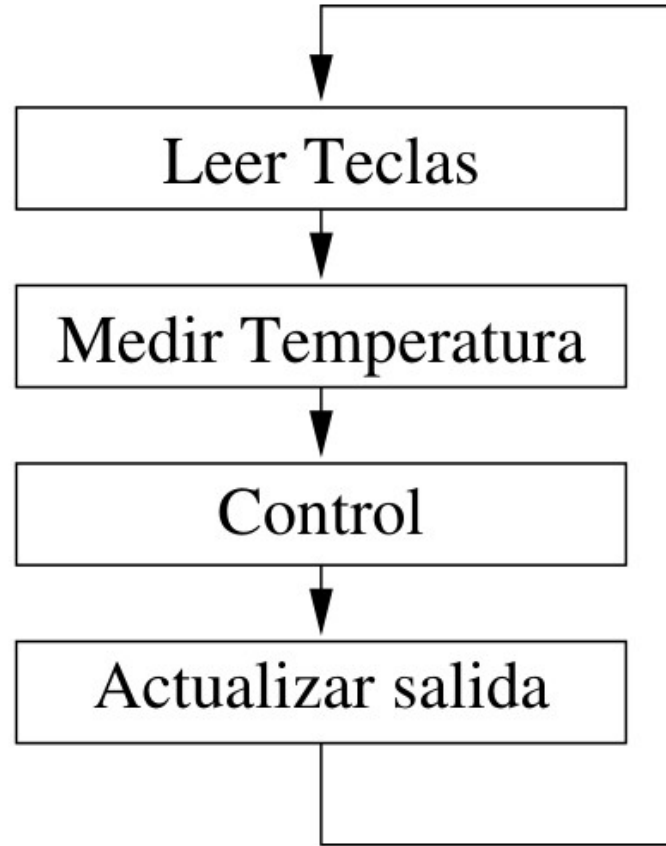


```
main()
{
    while(1){ /* Bucle infinito */
        Tarea1();
        /* Espera un evento externo */
        while(evento_externo == FALSO);
        /* ... */
        TareaN();
    }
}
```

Procesamiento secuencial (scan loop)



Ejemplo: Termostato



Ejemplo: Termostato

- Leer teclas:
 - Verifica si hay teclas pulsadas, en caso afirmativo modifica consigna. Si no hay teclas pulsadas, termina de ejecutarse (no bloquea).

```
tecla = LeerTeclado(); /* no bloquea */  
if(tecla == UP){  
    consigna ++;  
}  
if(tecla == DOWN){  
    consigna --;  
}
```



Ejemplo: Termostato

- Medir temperatura:
 - Lectura del sensor.
 - Conversión A/D.

```
LanzarConversionAD();  
while(Convirtiendolo());  
/* Esperar EOC */  
temp = LeerConversorAD();
```



Ejemplo: Termostato

- Medir temperatura:
 - Lectura del sensor.
 - Conversión A/D.

```
LanzarConversionAD();  
while(Convirtiendolo());  
/* Esperar EOC */  
temp = LeerConversorAD();
```



Ejemplo: Termostato

- Medir temperatura:
 - Lectura del sensor.
 - Conversión A/D.

```
LanzarConversionAD();  
while(Convirtiendolo());  
/* Esperar EOC */  
temp = LeerConversorAD();
```

- La tarea de conversión no es un evento asíncrono y el tiempo del conversor está acotado.



Ejemplo: Termostato

- Control:

- Control sencillo todo o nada o algún tipo de control proporcional más complejo, depende del tipo de salida.

```
if(temp < consigna){  
    calefaccion = ON;  
}else if(temp > consigna + HISTERESIS){  
    calefaccion = OFF;  
}
```



Ejemplo: Termostato

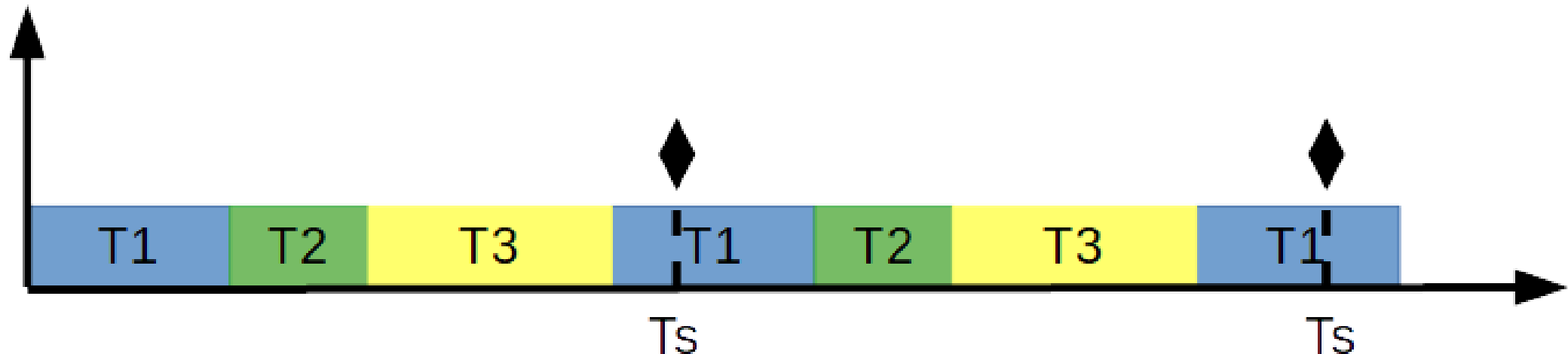
- Actualizar salida:
 - Enciende/apaga climatizador.
 - Actúa proporcionalmente en el equipo.

```
if(calefaccion){  
    activar_salida(0x01);  
    desactivar_salida(0x02);  
}else{  
    activar_salida(0x02);  
    desactivar_salida(0x01);  
}
```



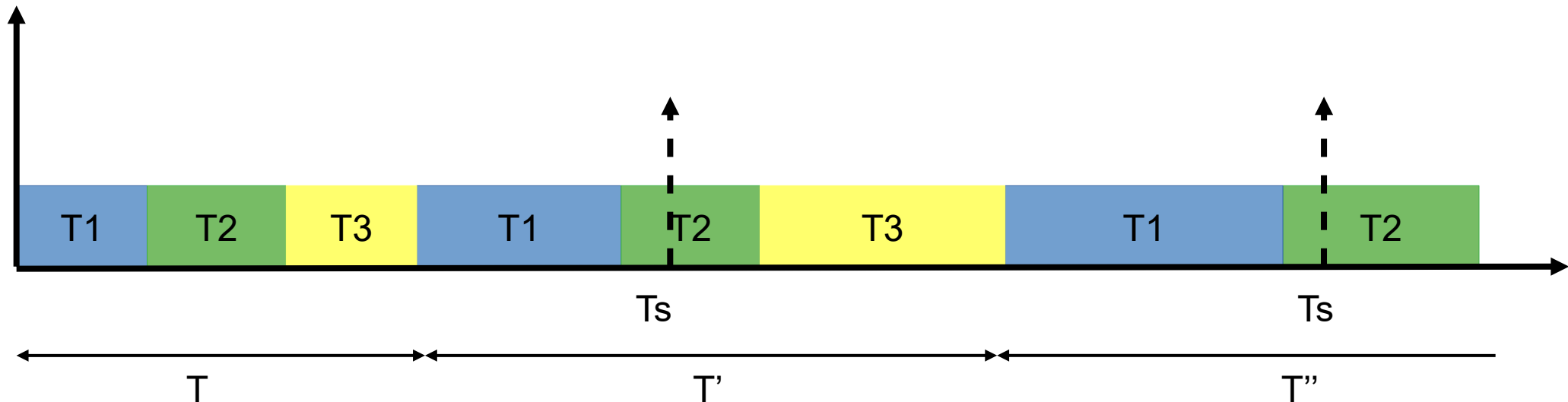
Temporización (scan loop)

- En muchos casos, se requiere un ciclo de scan (o tiempo de scan T_s) definido.
- No siempre esta condición se cumplirá para determinadas tareas.



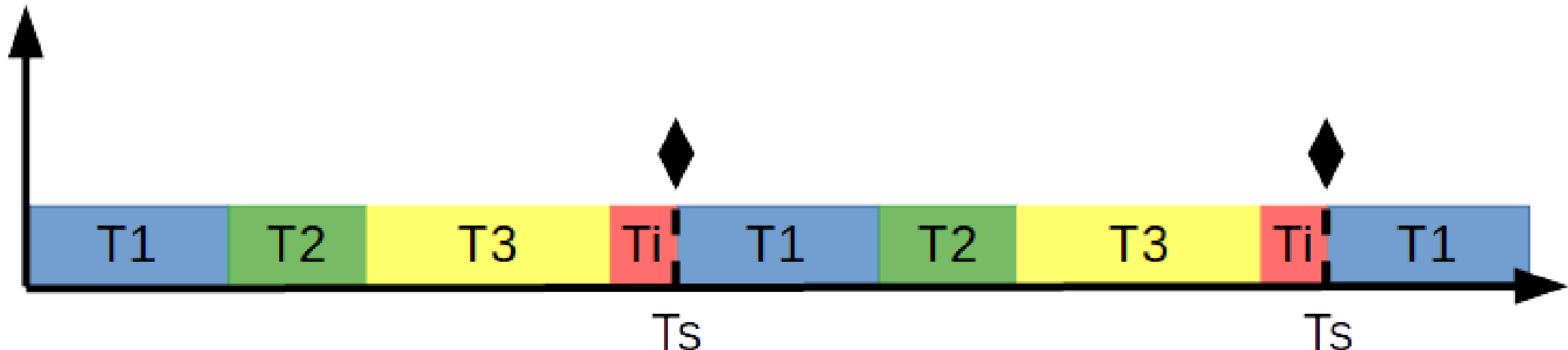
Temporización (scan loop)

- Incluso el período de ejecución de las tareas puede ser no ser homogéneo, lo que puede ser más problemático.



Temporización (scan loop)

- Este requerimiento se puede cumplir sumando un temporizador que ecualice la duración del ciclo.
- Esta tarea se suele denominar **ociosa o inactiva (idle)**



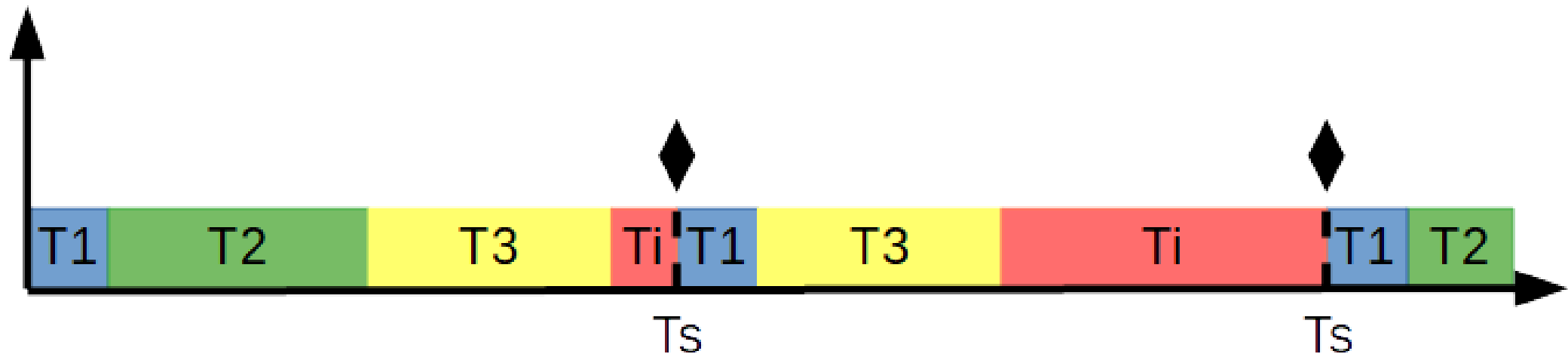
Temporización (scan loop)

- Esta funcionalidad se puede implementar mediante temporizadores, los cuales son habituales en los microcontroladores.

```
void TareaInactiva(void)
{
    while( ! fin_timer );
        /* Espera el final del periodo */
    ReinicializaTimer();
}
```

Tareas con distintos períodos (scan loop)

- Algunas tareas pueden requerir ejecutarse cada n ciclos de scan (período $n \times T_s$):



- **Si alguna tarea tiene período que no es múltiplo de T_s se debe recurrir a otro sistema (Foreground/Background, RTOS)**

Distintos períodos (scan loop)

- Alternativas:

```
n=0;
for(;;){ /*for-ever*/
    TareaA();
    if(n %2==0){
        TareaB();
    }
    n++;
    TareaC();
    TareaInactiva();
}
```

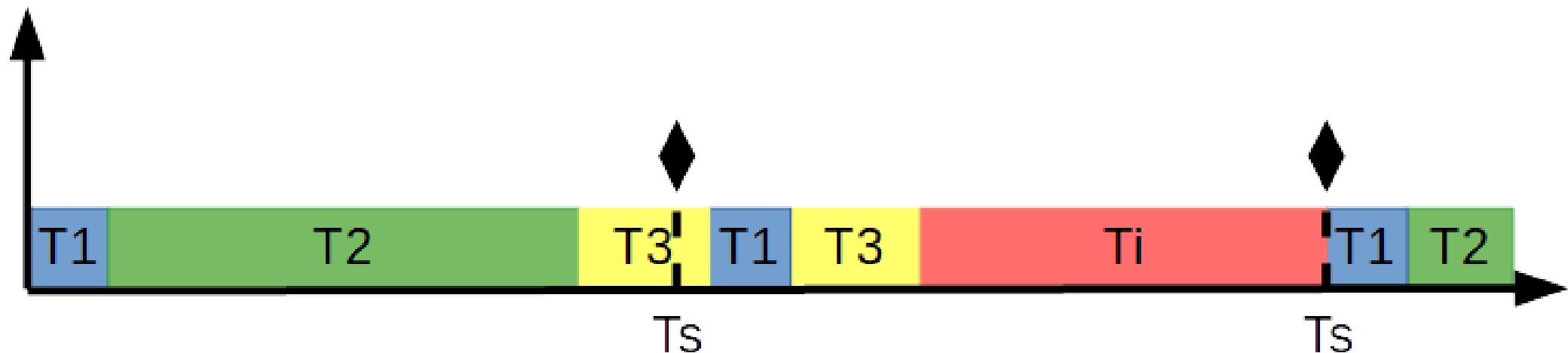


```
n=2;
for(;;){ /*forever*/
    TareaA();
    if(n==2){
        TareaB();
        n=0;
    }
    n++;
    TareaC();
    TareaInactiva();
}
```



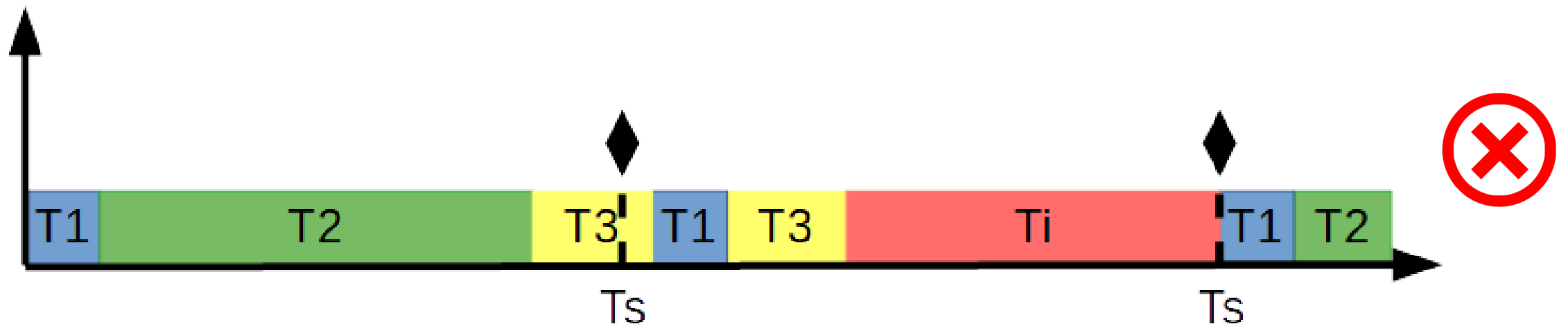
Tiempo de ejecución extenso (scan loop)

- El tiempo de ejecución excesivo representa un problema.
- Sin embargo, en ciertos casos se pueden realizar optimizaciones para el correcto funcionamiento.



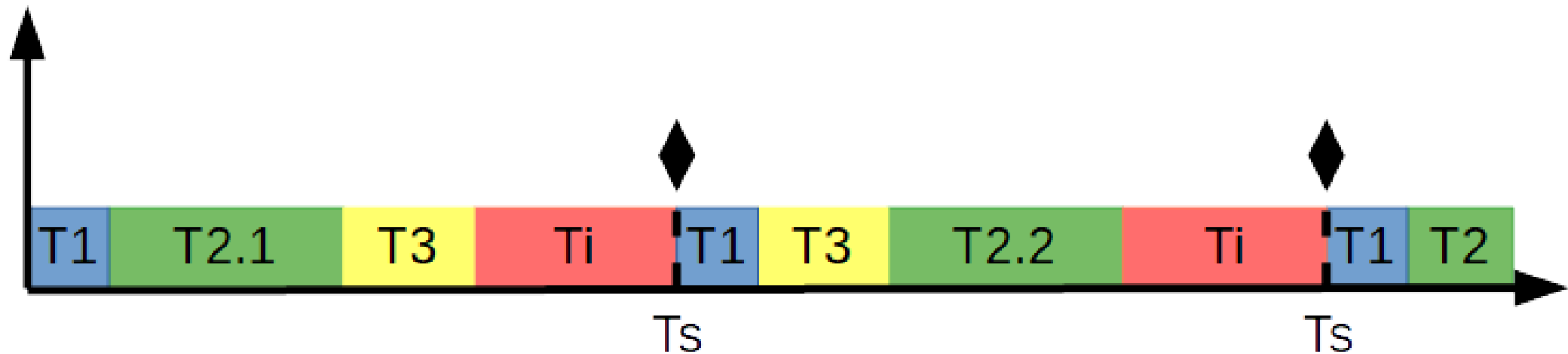
Tiempo de ejecución extenso (scan loop)

- El tiempo de ejecución excesivo representa un problema.
- Sin embargo, en ciertos casos se pueden realizar optimizaciones para el correcto funcionamiento.



Tiempo de ejecución extenso (scan loop)

- La solución más sencilla (y más costosa) es cambiar el procesador.
- En otros casos es posible dividir la tarea de larga duración en dos partes, aprovechando mejor el tiempo de procesamiento.



Tiempo de ejecución extenso (scan loop)

```
void TareaB(void)
{
    static int estado = 1;
    switch(estado){
    case 1:
        /* tareas B.1 */
        estado = 2;
        break;
    case 2:
        /* tareas B.2 */
        estado = 1;
        break;
    }
}

for(;;){
    TareaA();
    TareaB();
    TareaC();
    TareaInactiva();
}
```

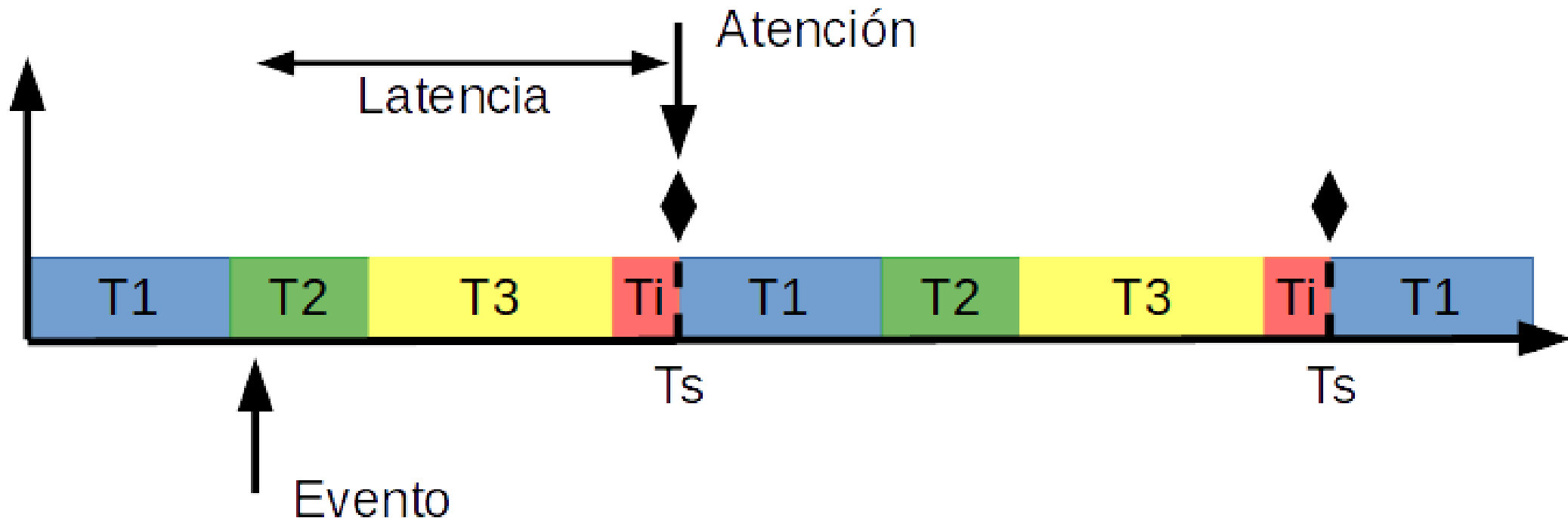
Latencia (scan loop)

La **latencia** en un sistema de tiempo real se define como el tiempo máximo que transcurre entre un evento (interno o externo) y el comienzo de la ejecución de la tarea que lo procesa.



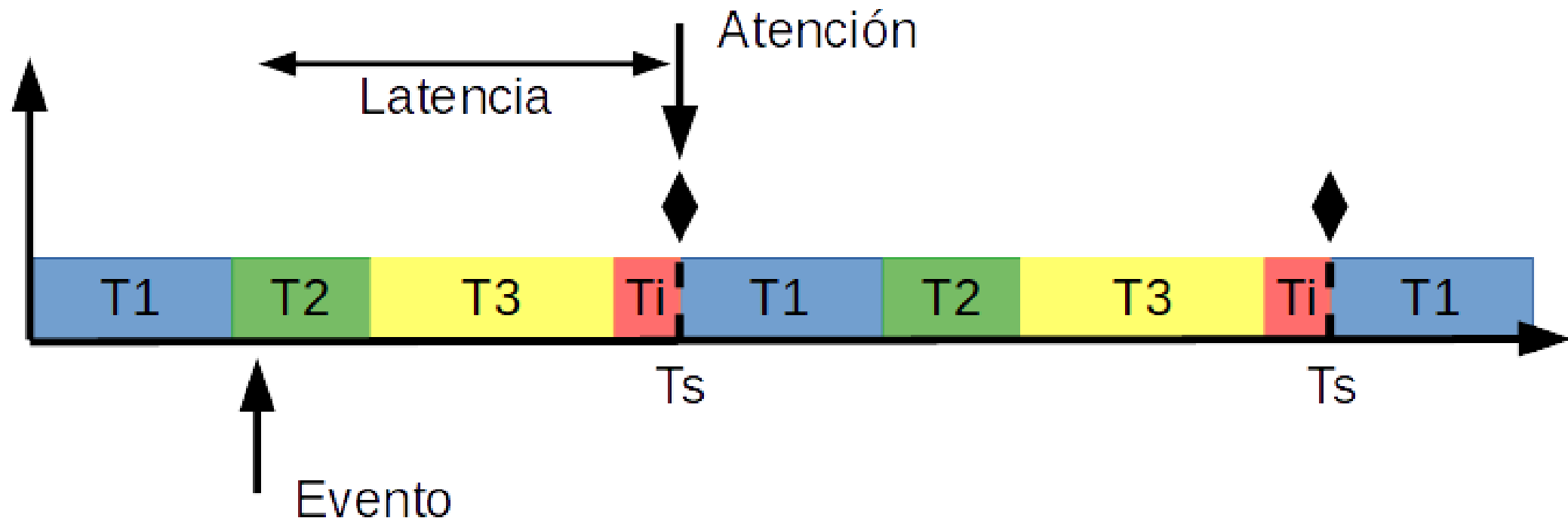
Latencia (scan loop)

- En el procesamiento secuencial, la latencia más alta puede ser igual a T_s



Latencia (scan loop)

- El peor caso es que el evento suceda justo después de que la tarea encargada de atenderlo lo verifique. La latencia en este caso es T_s



Ventajas e inconvenientes (scan loop)

- Facilidad de implementación y depuración.
- Facilidad para compartir datos.
- No utiliza tiempo para cambio de contexto.



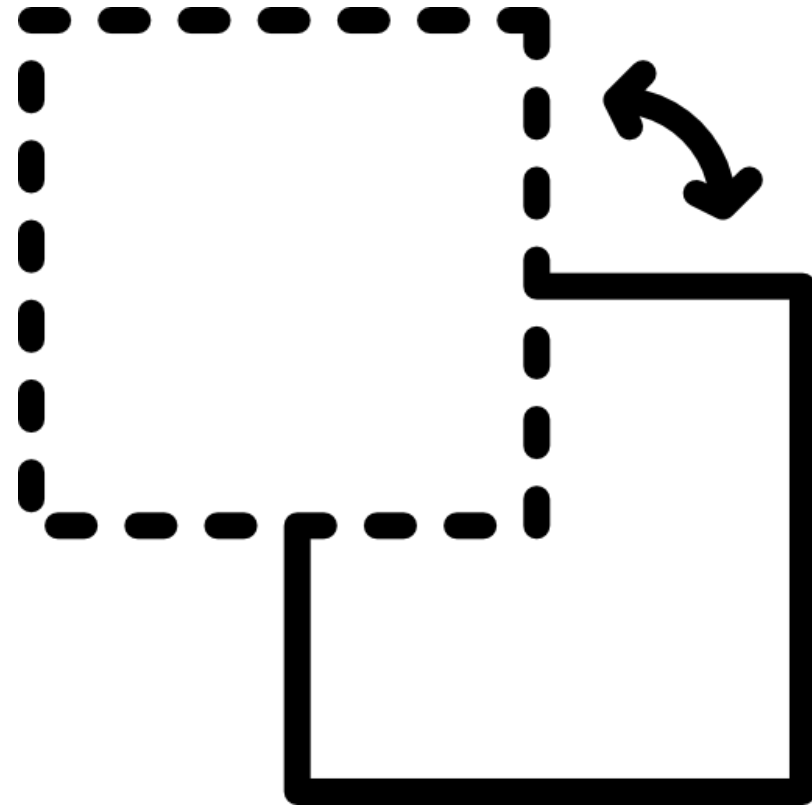
Ventajas e inconvenientes (scan loop)

- Latencia variable.
- Tareas con distintos períodos de muestreo.
- No escala para sistemas complejos.

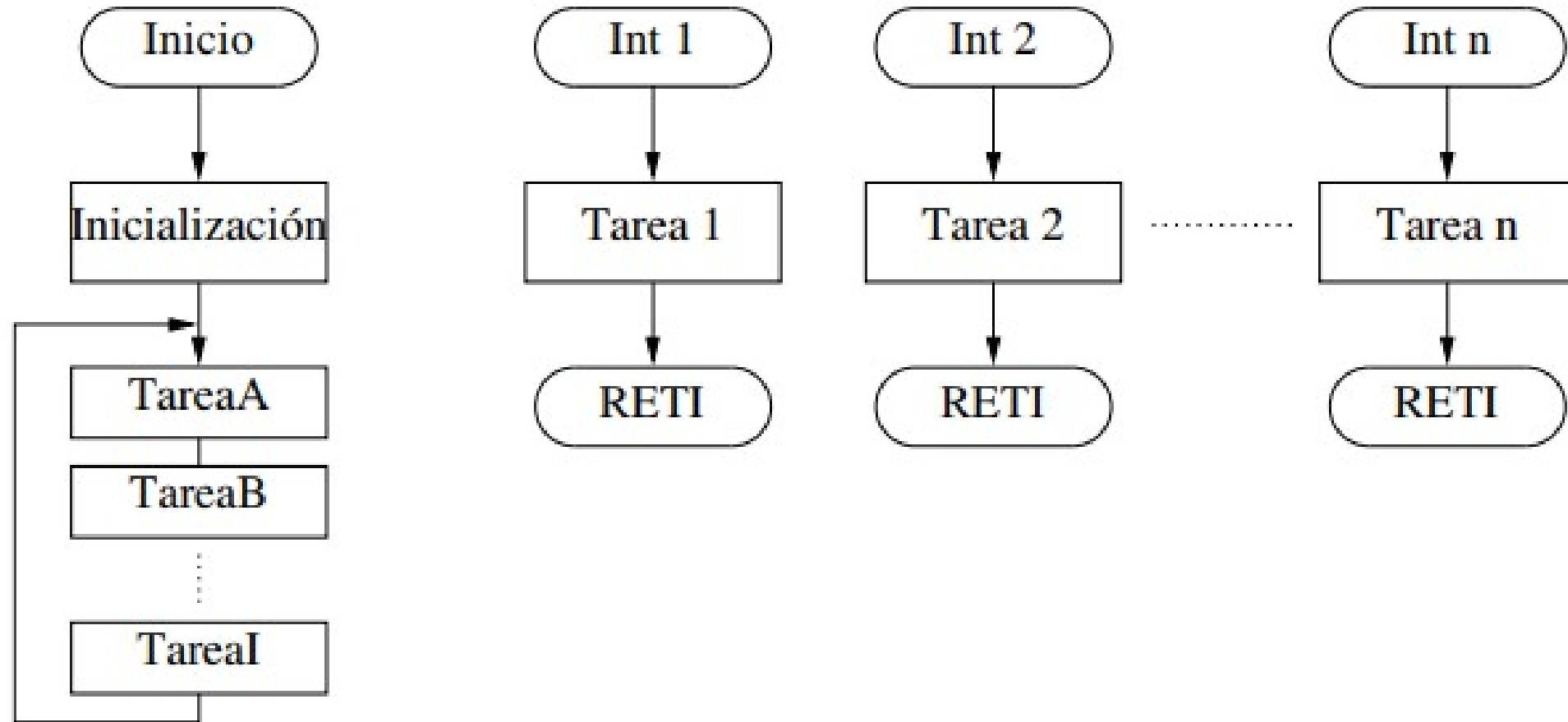


Foreground/Background

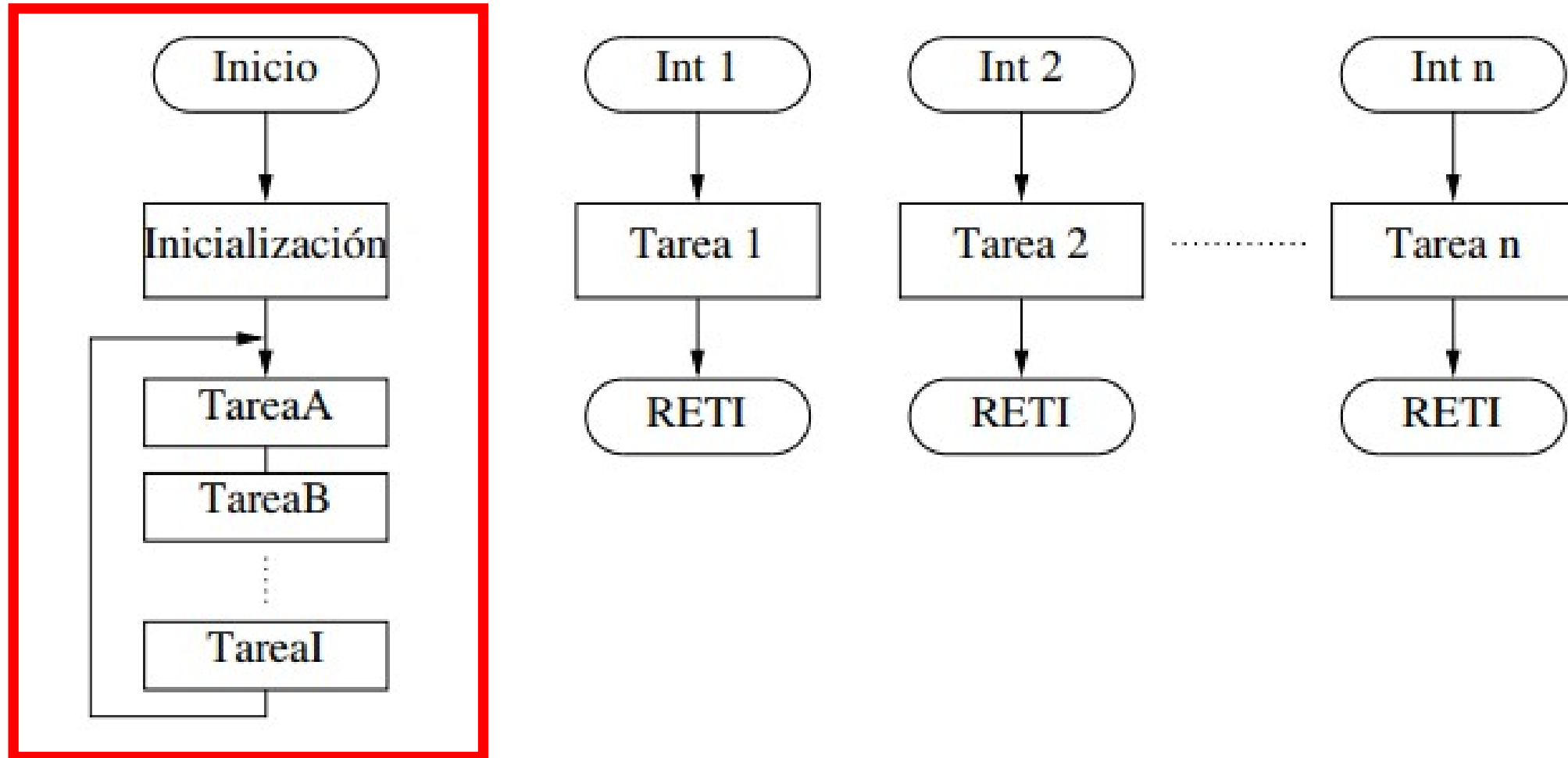
- Uno de los mayores problemas del scan loop es la latencia.
- Para disminuir este tiempo es posible asociar tareas a interrupciones.



Foreground/Background

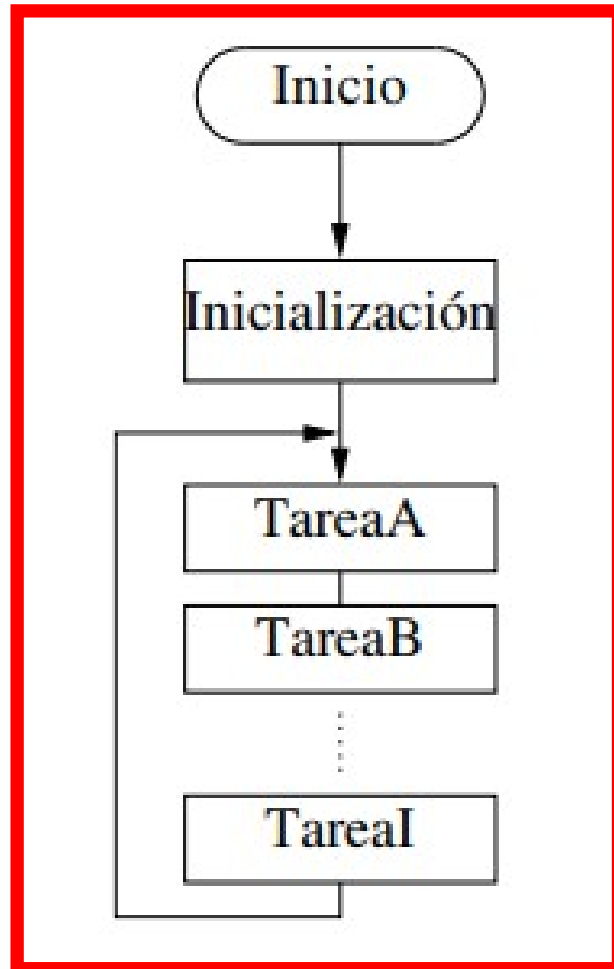


Foreground/Background

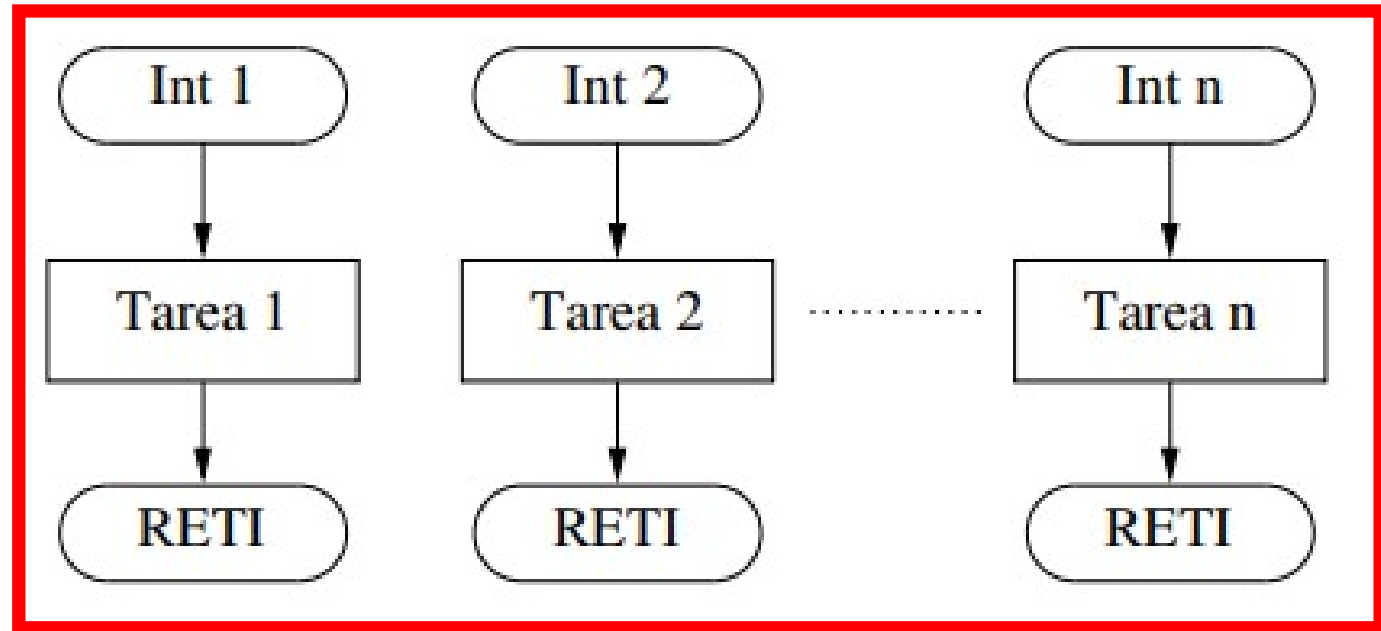


Primer Plano

Foreground/Background



Primer Plano



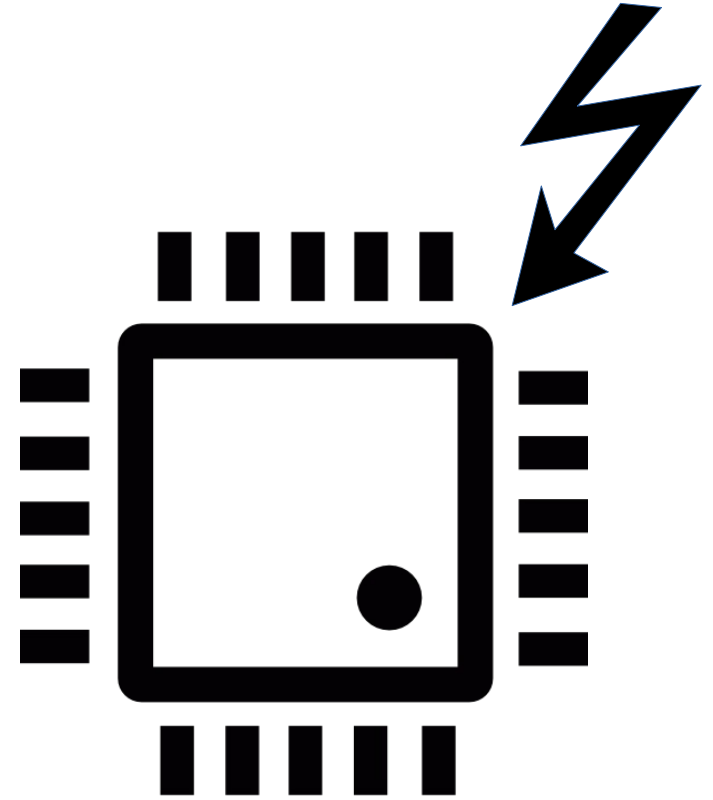
Segundo Plano

Foreground/Background

- Principales ventajas:
 - Baja latencia para la atención de tareas asociadas a interrupciones.
 - No requieren software adicional (RTOS).
- Inconvenientes:
 - Requieren soporte de interrupciones en el hardware.
 - Cada tarea background requiere una interrupción.
 - Más complejo de depurar, aparecen problemas de concurrencia de tareas.

Latencia F/B

- Como primera aproximación, se estima igual al tiempo máximo de ejecución de cualquier tarea de interrupción ya que en este tiempo, las interrupciones se encuentran deshabilitadas (las de menor prioridad al menos).
- Por lo tanto, las tareas en segundo plano deberían tener la menor duración posible.



Latencia F/B

```
void TareaTemp(void) /* Tarea de segundo plano */
{
    static int ms, seg, min, hor;

    ms++;
    if(ms == 1000){
        ms = 0;
        seg++;
        if(seg == 60){
            seg = 0;
            min++;
            if (min == 60){
                min = 0;
                hor++;
                if (hor == 24){
                    hor = 0;
                }
            }
        }
        ImprimeHora(hor, min, seg);
    }
}
```

- Tarea en segundo plano
- Tiempo de ejecución 20ms
- Tiempo de interrupción 1ms

Latencia F/B

```
void TareaTemp(void) /* Tarea de segundo plano */
{
    static int ms, seg, min, hor;

    ms++;
    if(ms == 1000){
        ms = 0;
        seg++;
        if(seg == 60){
            seg = 0;
            min++;
            if (min == 60){
                min = 0;
                hor++;
                if (hor == 24){
                    hor = 0;
                }
            }
        }
        ImprimeHora(hor, min, seg);
    }
}
```

- Tarea en segundo plano
- Tiempo de ejecución 20ms
- Tiempo de interrupción 1ms



Latencia F/B

```
void TareaTemp(void) /* Tarea de segundo plano */
{
    static int ms, seg, min, hor;

    ms++;
    if(ms == 1000){
        ms = 0;
        seg++;
        if(seg == 60){
            seg = 0;
            min++;
            if (min == 60){
                min = 0;
                hor++;
                if (hor == 24){
                    hor = 0;
                }
            }
        }
    }
}
```

- Tarea en segundo plano
- Tiempo de ejecución $\ll 1\text{ms}$
- Tiempo de interrupción 1ms

```
int main(void)
{
    InicializaTemporizador();
    HabilitaInterrupciones();

    for(;;){
        ImprimeHora(hor, min, seg);
        /* Resto de tareas de primer plano */
    }
}
```

- Tarea en primer plano

Latencia F/B

```
void TareaTemp(void) /* Tarea de segundo plano */
{
    static int ms, seg, min, hor;

    ms++;
    if(ms == 1000){
        ms = 0;
        seg++;
        if(seg == 60){
            seg = 0;
            min++;
            if (min == 60){
                min = 0;
                hor++;
                if (hor == 24){
                    hor = 0;
                }
            }
        }
    }
}
```

- Tarea en segundo plano
- Tiempo de ejecución $\ll 1\text{ms}$
- Tiempo de interrupción 1ms

```
int main(void)
{
    InicializaTemporizador();
    HabilitaInterrupciones();

    for(;;){
        ImprimeHora(hor, min, seg);
        /* Resto de tareas de primer plano */
    }
}
```

- Tarea en primer plano



Latencia F/B

```
void TareaTemp(void) /* Tarea de segundo plano */
{
    static int ms, seg, min, hor;

    ms++;
    if(ms == 1000){
        ms = 0;
        seg++;
        if(seg == 60){
            seg = 0;
            min++;
            if (min == 60){
                min = 0;
                hor++;
                if (hor == 24){
                    hor = 0;
                }
            }
        }
    }
}
```

- Tarea en segundo plano
- Tiempo de ejecución $\ll 1\text{ms}$
- Tiempo de interrupción 1ms

```
int main(void)
{
    InicializaTemporizador();
    HabilitaInterrupciones();

    for(;;){
        ImprimeHora(hor, min, seg);
        /* Resto de tareas de primer plano */
    }
}
```

- Tarea en primer plano
- ¿Qué tipo de variables son ms, seg, min, hor?
¿Qué visibilidad tienen?



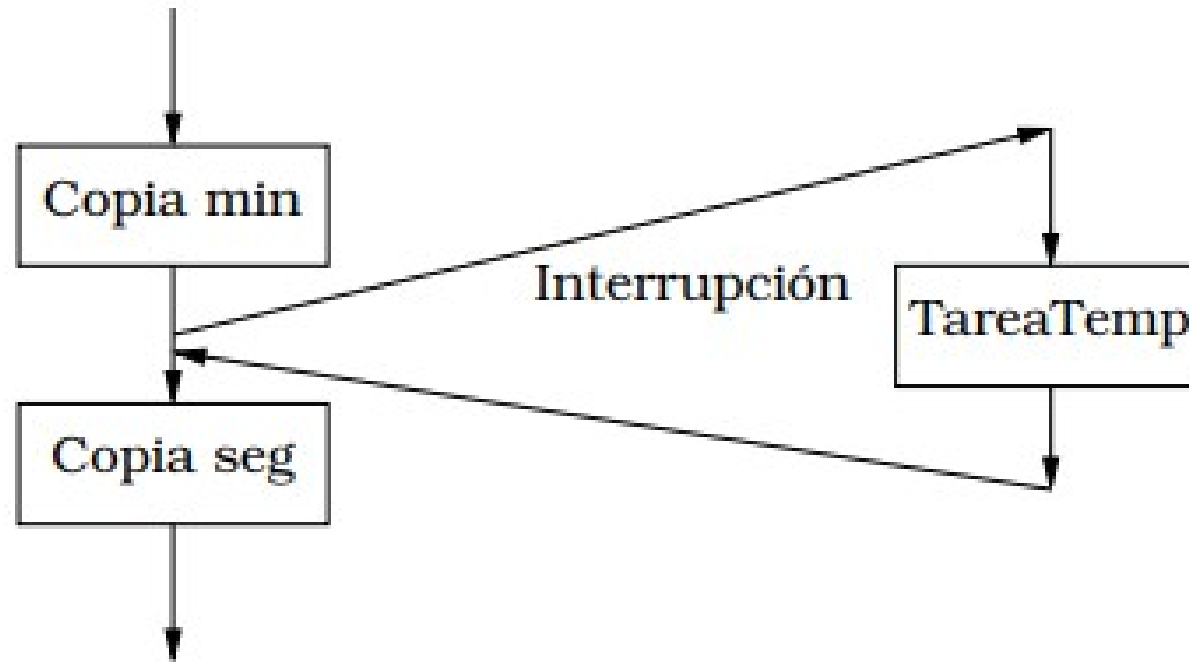
Datos compartidos F/B

- Al trasladar los procesos lentos a la tarea de primer plano se soluciona el problema de la latencia, pero por desgracia se crea un nuevo problema: **la comunicación entre tareas que se ejecutan asíncronamente.**



Datos compartidos F/B

- Se origina el problema de **incoherencia de datos**



Datos compartidos F/B

- Se origina el problema de **incoherencia de datos**

```
1  /* Type your code here, or load an example. */
2  #include <stdio.h>
3  int sum(int a, int b) {
4      return a + b;
5  }
6
7  int main(void)
8  {
9      int i = 2;
10     int j = 3;
11     int k;
12
13     k = sum(i, j);
14     return 0;
15 }
```

```
18  _main PROC
19      push    ebp
20      mov     ebp, esp
21      sub     esp, 12
22      mov     DWORD PTR _i$[ebp], 2
23      mov     DWORD PTR _j$[ebp], 3
24      mov     eax, DWORD PTR _j$[ebp]
25      push    eax
26      mov     ecx, DWORD PTR _i$[ebp]
27      push    ecx
28      call    _sum
29      add     esp, 8
30      mov     DWORD PTR _k$[ebp], eax
31      xor     eax, eax
32      mov     esp, ebp
33      pop     ebp
34      ret     0
35  _main ENDP
```

Datos compartidos F/B

- Se origina el problema de **incoherencia de datos**

```
ImprimeHora(hor, min, seg);
```

```
13:13:59
```

```
13:13:00
```

```
13:14:00
```

Datos compartidos F/B

- Zona crítica:
 - Se entiende por zona crítica la zona en la que se usan recursos compartidos entre dos tareas que se ejecutan de manera asincrónica.
 - Por ejemplo entre una tarea de primer plano y una de segundo plano, o dos de segundo plano.



Datos compartidos F/B

- Zona crítica:
 - Para evitar inconvenientes la ejecución de la zona crítica tiene que transitarse sin interrupción



Datos compartidos F/B

- Deshabilitación de interrupciones:

```
...  
for(;;){  
    InhabilitaInterrupciones();  
    ImprimeHora(hor, min, seg); /* Zona crítica */  
    HabilitaInterrupciones();  
}/* Resto de tareas de primer plano */  
...
```

Datos compartidos F/B

- Deshabilitación de interrupciones:

```
...  
for(;;){  
    InhabilitaInterrupciones();  
    ImprimeHora(hor, min, seg); /* Zona crítica */  
    HabilitaInterrupciones();  
}/* Resto de tareas de primer plano */  
...
```



Datos compartidos F/B

- Copia de variables:

```
int main(void)
{
    int chor, cmin, cseg; /* Copias */

    InicializaTemporizador();
    HabilitaInterrupciones();

    for(;;){
        InhabilitaInterrupciones();
        chor = hor; /* Principio de la zona crítica */
        cmin = min;
        cseg = seg; /* Final de la zona crítica */
        HabilitaInterrupciones();
        ImprimeHora(chor, cmin, cseg);
        /* Resto de tareas de primer plano */
    }
}
```

F/B

- Si una tarea de segundo plano es compleja, es necesario dividirla en dos partes, dejando la parte más costosa computacionalmente para el primer plano.
- Sin embargo, las tareas de primer plano se ejecutan mediante un bucle de scan, por lo que su latencia será el tiempo máximo de ejecución del bucle.
- Por tanto, en un sistema F/B plano, tan solo se soluciona el problema de la alta latencia si las acciones que requieren baja latencia son **simples**.

RTOS

- Para sistemas complejos:

Sistema Operativo de Tiempo Real



RTOS

- Componentes:
- Una serie de mecanismos para permitir compartir datos entre tareas, como por ejemplo colas FIFO
- Otra serie de mecanismos para la sincronización de tareas, como por ejemplo semáforos.
- Un planificador que decide en cada momento qué tarea de primer plano ha de ejecutarse.

